Project Report
Course Schedule App

Mengistu Animaw 1093890
Zekarias Jemberu 1093900
Michael Tsiway 1093909

Supervised By: Eng. Maha Yaghi

جامعة أبوظبي
ABU DHABI UNIVERSITY

Submission Date: June 10, 2025

Contents

1 Introduction **5**
1.1 Motivation 5
1.2 Problem Statement 6
1.3 Literature Review 6

2 Design **7**
2.1 Requirements, Constraints, and Considerations 7
2.2 Design Process 8
2.2.1 Design and Prototype Development 9
2.2.2 Refinement and Final Implementation 10
2.3 System Overview 11
2.4 Component Design 12
2.4.1 Authentication Module (Angular + Firebase) 12
2.4.2 Session Management Module 13
2.4.3 Load Calculation Engine 13
2.4.4 Conflict Detection Engine 14
2.4.5 Version Control and Comparison 14

3 Experimental Testing and Results **14**
3.1 Testing Plan and Acceptance Criteria 14
3.1.1 Test Case 1: User Signup and Login 15
3.1.2 Test Case 2: Session Creation and Grid Placement . . 15
3.2 Test Case 3: Instructor Load Color Coding 16
3.3 Results 16
3.3.1 User Interface Functionality 16
3.3.2 Firestore Functionality 20
3.4 Analysis and Interpretation of Data 22
3.4.1 System Accuracy and Behavior 22
3.4.2 Conflict Detection Insights 22
3.4.3 Version Control Comparison 22
3.4.4 Export options 22
3.4.5 Procedure Quality and Recommendations 23

4 Conclusion **23**
4.1 Summary 23
4.2 Future Improvements and Takeaways 23

4.3 Lessons Learned 24
4.4 Team Dynamics 24
4.5 Impact Statement 25

List of Figures

1 The user interface of the course schedule app for sign up and log in purpose. 9
2 The user interface of the dash board. 17
3 The user interface of the Schedule view app component on a Samsung phone. 18
4 The add and drop enabling form of the course schedule app . . 18
5 The user interface of the load calculation app component on a Samsung phone. 19
6 The user interface of the error table component on a phone. . 19
7 The user interface of the version comparing app feature. . . . 20
8 A sample session saved in Firestore. 21
9 The version names and objects saved under them in Firestore. 21

List of Tables

Abstract

In this project, the design and implementation of a schedule application for use by students to help them organize our university course sessions effectively is presented. The main intention is to facilitate the process of assigning course sections to instructors or coordinators by presenting the interface for the instructors or coordinators to define course offerings, assign instructors to courses, allocate the sessions to predefined time slots, and manage unique campus schedules for Abu Dhabi and Al Ain. It has key features such as a dual-grid visual timetable for MW and TR slots, dynamic session creation and deletion along with Firebase integration, automated faculty workload calculation, conflict detection, version comparison tools, etc... Real-time workload imbalances and schedule conflicts are nicely highlighted using real-time color-coded tables. It extends usability support with versioning support, which allows schedule comparison and incremental improvement. The implementation is proven scalable, where the platform can be embedded within a process in a highly interactive manner and leads to optimal process scheduling in an accurate, persistent fashion.

1 Introduction

Efficient course scheduling is essential for academic institutions to ensure fair faculty workload distribution and timely student access to courses. Manual scheduling methods are often time-consuming, error-prone, and difficult to adapt—especially across multi-campus systems like Abu Dhabi and Al Ain. Given the subscription nature of academic institutions, where the students pay for the courses on a quarterly, semesterly, or yearly basis, fair faculty workload distribution and timely student access to courses are required. Scheduling manually is oftentimes slow, error-prone, and challenging to adapt to across the multi-campus systems that we have in the Abu Dhabi and Al Ain campuses. Specifically, the application introduced in this project is a digital scheduling software. Users are able to create course sections, attach instructors, and drop sessions into pre-defined time slots by using a visual grid. The system is built with Firebase integration and allows real-time synchronization, persistent storage, and collaborative access to the data. This application is different from generic tools because it has specific fea- tures like instructor load tracking, cross campus conflict detection and version comparison. The main goal of the solution is its automation of the academic scheduling, decrease of the number of errors and making the planning more transparent.

1.1 Motivation

Scheduling departments' courses across multiple campuses is challenging, and they often cannot maintain fair and conflict-free schedules. Instructors are overloaded, resources are misused, and mistakes in scheduling are made to the detriment of teaching and learning. Manual or out-of-date systems do not offer the necessary flexibility and responsiveness of today's higher academics. However, these issues are addressed through this project, which develops an innovative, cross-platform sort scheduling app that takes care of the work automatically, cuts down errors, and facilitates a balanced workload for the faculty. The result is a better, faster, more transparent, and more adaptable scheduling reality and experience for administrators and educators.

1.2 Problem Statement

Scheduling courses across multiple campuses in academic departments is a problem that is hard to handle efficiently, such that faculty workload distribution is as fair as possible and departments are treated as a unit without session conflicts. This is not only time-consuming and error-prone but also lacks real-time coordination, resulting in double bookings, overloads, and inconsistent ties between campuses. Since there is no centralized, interactive system, it is difficult to visualize schedules, track changes, and compare different versions. This project is a solution to these issues that is based on developing a digital scheduling application that automates session assignment, conflict detection, faculty load calculation, and version management.

1.3 Literature Review

The challenge of creating effective course scheduling systems has driven research in both academic and industrial contexts. Traditionally, they resort to using rule based logic or optimization algorithms that are computationally expensive and not flexible to real–time changes. Today's systems must support real time collaboration, cross platform compatibility and scalability to help meet the institution's need.

Tokheim [1] argues for logical design in digital systems, e.g. in the design of control logic for applications such as automated schedulers. He offers the theory of rule driven design of time slot and load management. Similarly, Givone [2] involves concentration on digital design principle that facilitate in modular design of the system for ease of scalability and maintainability of the logic components of the app.

In this light perspectives from Systems engineering such those discussed by Blanchard and Fabrycky in [3] provide some insight into the lifecycle design and user driven development which are critical to educational tools. Iterative development and requirements refinement which are mainstays of version management and conflict resolution in this application, are supported by their model.

The development guidance in Hazzard and Hazzard [4] describe how they are using Firebase in real world software projects and support modern cloud and real time database integration which Firebase provides. Live data sync, authentication and deployment scalability in educational and administrative tools are shown to be important through their work.

In addition, Sommerville's work in software engineering [5] presents the requirements for requirement analysis, modularization and testing, providing all information that makes up the backend logic and frontend development practises of this scheduling app. He focuses on a world that is in constant change and evolution which is exactly what the versioning and comparison features implemented in this project are based on.

To summarise, although such platforms provide crucial building blocks like the toolset for scheduling, they generally lack real time adaptability and do not work for specific campuses. This project intends to close this gap using modern cloud integration, interactive UI and a system architecture driven by the guidance of modern systems and software engineering principles.

2 Design

2.1 Requirements, Constraints, and Considerations

Design Requirements and Constraints

The following requirements and constraints were identified based on project statements and user expectations:

- **Time Slot Grid:** The application must use a fixed grid structure of 14 slots (7 for MW, 7 for TR), each with predefined time intervals.
- **Dual Campus Support:** The system must differentiate between Abu Dhabi and Al Ain campuses and maintain separate schedule grids for each.
- **Course Section Creation:** Users must be able to create sections by selecting a course, instructor, section number, capacity, slot, and campus.
- **Real-Time Updates:** All changes should reflect instantly using Firebase Firestore Database to ensure live synchronization and data persistence.
- **Faculty Load Tracking:** The system must calculate and display the load per faculty member with visual indicators (green, yellow, red).

- **Conflict Detection:** Conflicts such as instructor double-booking must be automatically detected and listed in an error table.
- **Version Saving and Comparison:** Users must be able to save multiple schedule versions and compare them to identify changes.
- **Responsive UI:** The interface must be user-friendly and accessible across desktop and mobile platforms.
- **No Overwriting:** Adding a new section must not overwrite existing sessions in the same slot; the system should prevent such actions.

Public, Social, and Global Considerations

The design of the system also accounts for broader public, safety, and ethical considerations, including:

- **Public Health and Well-Being:** Ensures instructors are not overloaded, promoting mental well-being and avoiding burnout.
- **Safety Consideration:** Helps reduce human error in scheduling, which can lead to class conflicts and unnecessary campus travel.
- **Social and Cultural Factors:** Supports scheduling across campuses in culturally diverse regions; considers local norms such as weekend patterns.
- **Environmental Considerations:** Reduces paper-based scheduling and reprinting, contributing to environmentally sustainable practices.
- **Economic Factors:** Automates a manual process, saving time and resources for administrative staff and reducing operational costs.
- **Impact at large:** The modular design can be adapted to other institutions or multi-campus models, making it significantly applicable.

2.2 Design Process

The process for development was an iterative design process made of planning, prototyping, testing, and refining. First, we made a functional prototype using Angular, and then we integrated it with the Firebase Firestore database.

Basic features such as session creation, grid display, and instructor assignment were implemented. After internal testing, I finally figured out problems such as improper conflict detection and responsiveness to UI. Based on feedback, the form validations were improved, the schedule grid was optimised, and real-time data handling was optimised. The end design tracked faculty loads, supported dynamic session updates, error detection, and version comparison, and completed the project's requirements, with the flexibility to accommodate future ex-tensions.

2.2.1 Design and Prototype Development

In the design process, we first brainstormed about the final user interface and functionality, the database to back up the data, and the process to achieve it. So, we decided to use Angular as a development tool, five features to do the required tasks, and Firebase to store our data permanently, including control of accounts to sign in. Figure 1 shows the first user interface and features developed, including the account creation and sign-in prototype.

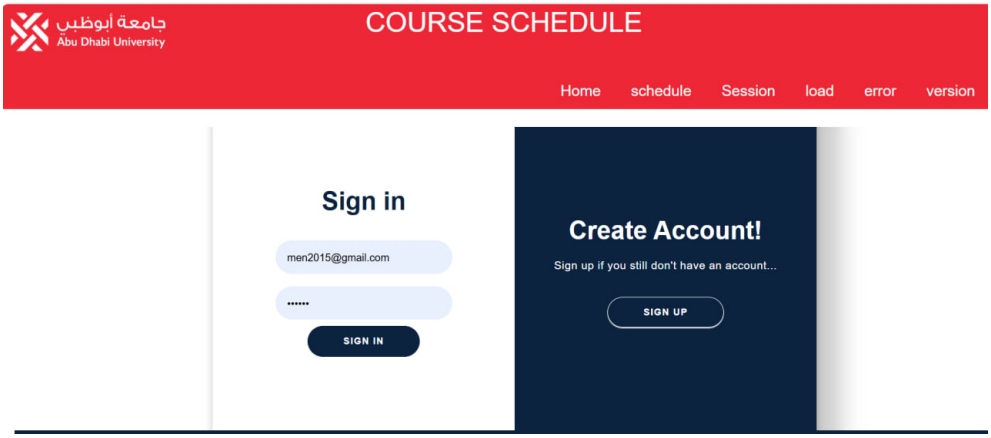

Figure 1: The user interface of the course schedule app for sign up and log in purpose.

The prototype for session creation, visual grid placement, and secure auauthentication was the first one. We first started by building form components for having fields to enter course section details (course name, instructor, section number, slot, campus, etc.). To test live data synchronization and data retrieval, these entries were connected to the Firebase Real-time

Database. We then implemented the timetable grid layout in which sessions can be rendered dynamically according to user inputs. Campuses and time slots were distinguished through visual feedback and styling. We also implemented instructor load tracking and a basic error table for conflict detection of this aspect to serve as a basis for more development. Once we had created the first prototype, we went ahead and performed usability testing and functional bugs testing. Feedback from the colleagues was used to better validate the form, handle errors better and make the interface more responsive on multiple devices. As a result of these improvements we have a stable and functional scheduling system as presented in the final version.

2.2.2 Refinement and Final Implementation

After the initial prototype, several refinements were made based on functionality testing and user feedback. The improvements included empowering the conflict detection mechanism. Some additional rules to detect scheduling conflicts were also introduced at both the same and multiple campus levels, where we were particularly concerned about detecting scheduling conflicts of instructors among multiple sessions in the same time window. We also improved the faculty load calculation feature by creating a color-coded workload table that keeps updating in real time as sessions are added or removed. It help users to quickly find underloaded, balanced and overloaded instructors thus making schedule management more efficient. The timetable was optimized visually so that it would be as clear and scalable as possible. For this the spacing, labels and the dynamic rendering were adjusted to work with changes in screen sizes supporting the application to work for almost any device. Some additional improvements were made by incorporating a version control component, which lets users save various versions of the schedule and review them as desired. With this, the iterative planning was made flexible, and the collaborative scheduling decisions were supported. We finally secured the application by ensuring Firebase Authentication for sign-in and restricting access to all admin features. The final implementation integrated all core features, including session creation and deletion, real–time link to the database, workload tracking, error detection, and version comparison, into one effective and user–friendly system.

2.3 System Overview

The course scheduling system is built using a modular Angular frontend and Firebase backend services. The system is organized into three main layers: the User Interface (UI), the Service Layer, and the Cloud Backend.

1. Frontend Layer (Angular UI): The UI is composed of Angular components responsible for handling user input, displaying schedules, managing sessions, and rendering instructor load summaries. The key components include:

- `LoginComponent`: Manages user sign-in and sign-up.
- `HomeComponent`: Central dashboard for navigation.
- `SessionComponent`: Enables session creation and deletion.
- `ScheduleComponent`: Displays structured schedules for Abu Dhabi and Al Ain campuses.
- `LoadComponent`: Summarizes teaching loads by instructor.
- `ErrorComponent`: Displays scheduling conflicts and warnings.
- `VersionComponent`: Handles schedule versioning and comparison.

2. Authentication Layer: Authentication is managed by the `AuthService`, which wraps Firebase Authentication. It supports user registration, login, logout, and session persistence. Authentication status is tracked using local storage and monitored in real time.

3. Service Layer (PersistenceService): This service provides a unified API to interact with Firestore. It handles CRUD operations for session data, instructor load calculations, conflict detection, and version management. All operations are asynchronous and update the UI dynamically.

4. Backend (Firebase): Firebase serves as the backend platform offering:

- **Authentication:** Secure email/password-based sign-in.
- **Firestore Database:** Stores sessions, versions, and real-time updates.

System Flow: Users sign up or log in using the login interface. Upon authentication, they access system features through the dashboard. User actions trigger methods in the `PersistenceService`, which interact with Firestore to fetch or update data. All changes are reflected in real time across the UI.

2.4 Component Design

This section describes the main subcomponents of the scheduling application. It includes code snippets, figures, and structural details for the most critical features of the system.

2.4.1 Authentication Module (Angular + Firebase)

This module enables users to sign up and log in using Firebase Authentication. The interface dynamically switches between Sign-Up and Sign-In views.

Sign-Up Function:

Listing 1: Sign-Up Method in login.component.ts

```
signup() {
  this.authService.signup(this.email, this.password)
    .then(() => {
      return signOut(this.ngAuth);
    })
    .then(() => {
      this.signupMessage = 'Account created. Please sign in.';
      this.isSignUp = false;
      this.email = '';
      this.password = '';
    })
    .catch(error => alert('Sign Up Error: ' + error.message));
}
```

HTML Input Fields:

Listing 2: Sign-Up Form Fields

```
<form (ngSubmit)="signup()">
```

```
  <input type="email" [(ngModel)]="email" name="signupEmail"
      required />
  <input type="password" [(ngModel)]="password"
      name="signupPassword" required />
  <button type="submit">Sign Up</button>
</form>
```

2.4.2 Session Management Module

This module handles creation and deletion of course sections based on slot, instructor, and campus.

Firestore Add Section Example:

Listing 3: Add Section in PersistenceService

```
addSection(section: SectionModel): Promise<void> {
  const docRef = doc(this.firestore, `sections/${section.id}`);
  return setDoc(docRef, section);
}
```

Delete Section:

Listing 4: Delete Section by ID

```
deleteSection(id: string): Promise<void> {
  const docRef = doc(this.firestore, `sections/${id}`);
  return deleteDoc(docRef);
}
```

2.4.3 Load Calculation Engine

This component counts the number of assigned sections for each instructor and flags overload/underload.

Instructor Load Map:

Listing 5: Instructor Load Calculation

```
calculateLoad(sections: SectionModel[]): Map<string, number> {
  const loadMap = new Map<string, number>();
  sections.forEach(sec => {
```

```
    loadMap.set(sec.instructor, (loadMap.get(sec.instructor) || 0)
        + 1);
  });
  return loadMap;
}
```

2.4.4 Conflict Detection Engine

This engine checks for instructors double-booked at the same time.

Sample Time Conflict Check:

Listing 6: Time Conflict Detection

```
hasTimeConflict(section1: SectionModel, section2: SectionModel):
    boolean {
  return section1.slot === section2.slot &&
         section1.instructor === section2.instructor &&
         section1.campus !== section2.campus;
}
```

2.4.5 Version Control and Comparison

This module saves and compares different scheduling states.

Save Version Function:

```
saveVersion(schedule: any): Promise<void> {
  const versionId = `version_${Date.now()}`;
  const docRef = doc(this.firestore, `versions/${versionId}`);
  return setDoc(docRef, { timestamp: new Date(), data: schedule });
}
```

3 Experimental Testing and Results

3.1 Testing Plan and Acceptance Criteria

The following test cases were designed to evaluate the core functionality of the course scheduling system. Each test case includes a clear description,

test steps, expected and observed outcomes, acceptance criteria, and the final result.

3.1.1 Test Case 1: User Signup and Login

Test Name: User Authentication
Test Description: This test verifies that a user can sign up with an email and password, and then log in using those credentials.
Test Steps:

1. Navigate to the login page.
2. Enter a new email and password.
3. Click "Sign Up".
4. After successful sign-up, sign out.
5. Re-enter the same credentials and click "Sign In".

Expected Results: Account is created and login redirects user to the dashboard.
Observed Results: The user was able to create an account and log in successfully.
Acceptance Criteria: New users can create accounts and access the dashboard after login.
Test Result: Pass

3.1.2 Test Case 2: Session Creation and Grid Placement

Test Name: Add Session
Test Description: This test checks if a user can add a new course section, and whether the session appears in the correct slot.
Test Steps:

1. Navigate to the "Add Section" form.
2. Select a course, instructor, slot, capacity, and campus.
3. Click "Add Section".
4. View the session grid to confirm placement.

Expected Results: A new session appears in the grid under the specified slot and campus.
Observed Results: The section was successfully added and displayed in the correct grid cell.
Acceptance Criteria: Section details are stored in Firestore and displayed correctly in the schedule view.
Test Result: Pass

3.2 Test Case 3: Instructor Load Color Coding

Test Name: Load Indicator Accuracy
Test Description: This test ensures that instructors' workloads are updated and color-coded accurately based on session count.
Test Steps:

1. Assign varying numbers of sessions to three instructors.
2. Observe the instructor load table.

Expected Results: Instructors with 4 sessions show yellow, under 4 green, and over 4 red.
Observed Results: Color codes matched the expected ranges: Menge (5 - red), Zakir (2 - green), Amir (4 - yellow).
Acceptance Criteria: Color coding and session count display align with load thresholds.
Test Result: Pass

3.3 Results

This section summarizes the outcomes and performance of the developed course scheduling system. The results are divided into two main categories: User Interface functionality and Firestore database functionality.

3.3.1 User Interface Functionality

The system provides a fully interactive and responsive user interface for both administrators and instructors. Key features successfully implemented include:

- **Authentication Interface:** The sign-up and login components allow users to securely create and access accounts using Firebase Authentication. Upon successful login, users are redirected to the main dashboard.

- **Dashboard Navigation:** The dashboard provides intuitive navigation to various modules, including schedule view, session management, error table, version history, and instructor load as shown in Figure 2.

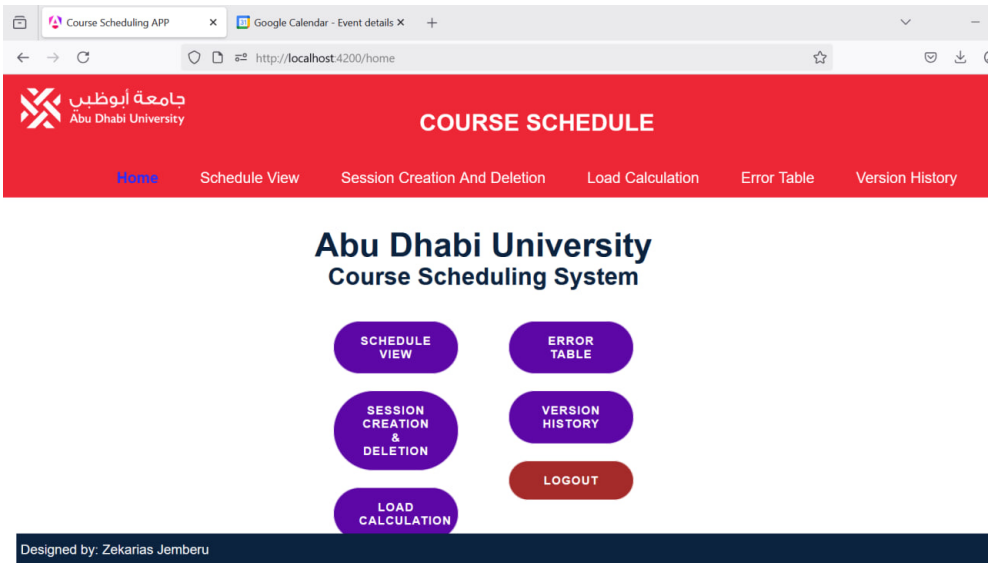

Figure 2: The user interface of the dash board.

- **Schedule View:** component shows the grids with 14 timeslots (7 MW and 7 TR) in each campus (Abu Dhabi and Al Ain). The portion of it is shown in Figure 3. The user can drop a course by using the X button, which is colored red. The exports to Excel and to Google Calendar are also integrated here.

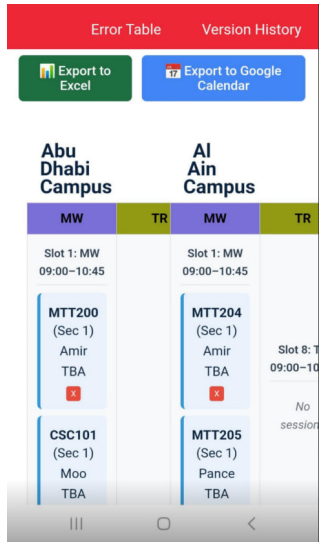

Figure 3: The user interface of the Schedule view app component on a Samsung phone.

- **Session Management UI:** Users can easily add or delete course sections using structured forms shown in Figure 4. Each session includes attributes such as course, instructor, capacity, slot, and campus. Sessions are rendered into the correct time-slot grid.

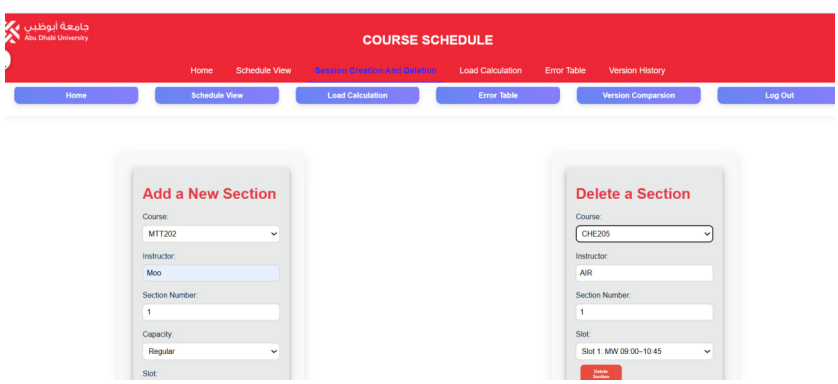

Figure 4: The add and drop enabling form of the course schedule app

- **Instructor Load Summary:**A dynamically updating table shows the number of sessions assigned to each instructor, using a clear color-

coding system to indicate underload, balanced, and overload cases. Since we want to develop an app with cross-platform functionality, we run our code on different devices. Figure 5 displays one screenshot from a phone for load calculation.

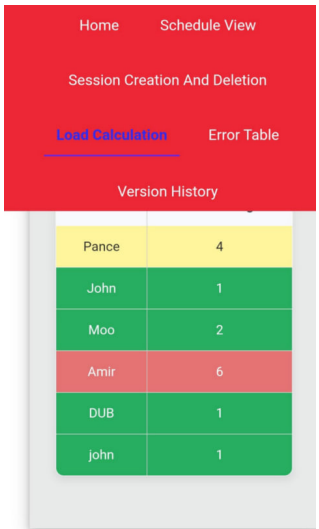

Figure 5: The user interface of the load calculation app component on a Samsung phone.

- **Error Table:** This table shows the conflicts in time in red and courses given only on one campus in yellow color, as shown in Figure 6

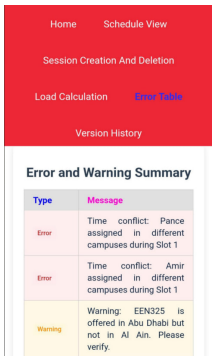

Figure 6: The user interface of the error table component on a phone.

- **Version Comparison UI:** Users can save schedule versions and compare differences between two versions using a dropdown and visual difference list as depicted in Figure 7.

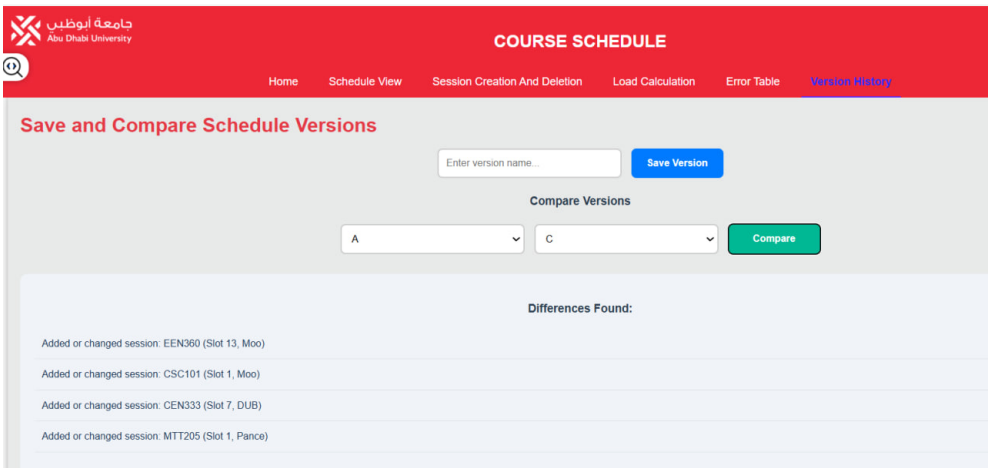

Figure 7: The user interface of the version comparing app feature.

3.3.2 Firestore Functionality

The application successfully integrates Firebase Realtime Database and Firestore for persistent data storage and synchronization. The following functionalities were confirmed:

- **Session Data Storage:** Each session created is saved with structured fields such as course, instructor, slot, capacity, and campus in Firestore as shown in Figure 8. The correct nesting of documents within collections like `sessions` and `scheduleVersions` was verified.

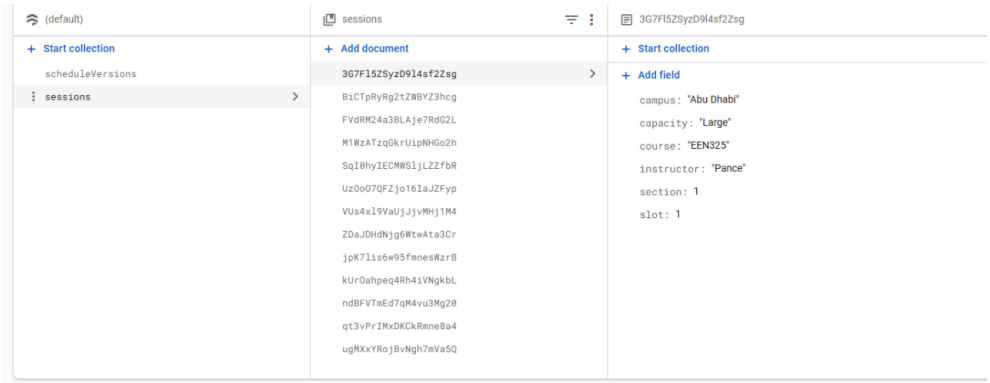

Figure 8: A sample session saved in Firestore.

- **Versioning System:** Schedule versions are saved with metadata such as version name and timestamp as displayed in Figure 9. Each version includes an array of all sessions at the time of saving, allowing future comparisons.

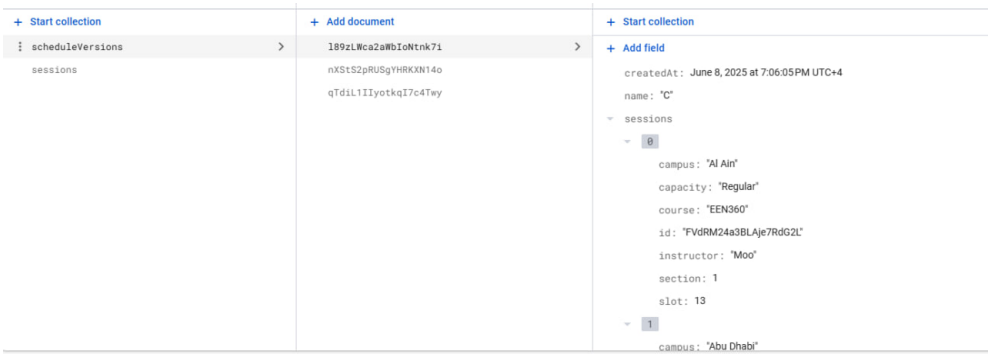

Figure 9: The version names and objects saved under them in Firestore.

- **Data Synchronization:** The UI reflects changes to the Firestore database in real-time, allowing collaborative usage and consistent updates across different views and components.
- **Data Accuracy and Querying:** Queries such as version comparison and instructor load retrieval are performed efficiently. Data retrieved from Firestore is displayed correctly in all relevant UI sections without latency or inconsistency.

3.4 Analysis and Interpretation of Data

The analysis of the data obtained from the course scheduling application highlights several important aspects of its functionality and effectiveness.

3.4.1 System Accuracy and Behavior

The system successfully saved, loaded, and updated session data across both campuses using Firestore. Visual verification of session display and instructor load calculations confirmed that entries were stored in the correct format, matched the selected input values, and appeared correctly in the MW/TR grid views. Minor inconsistencies in load display were initially observed due to mis-assigned slot numbers, which were resolved by casting slot values to integers before processing. Once corrected, the instructor load summary reflected accurate color-coded classification (underloaded, balanced, overloaded), validating the logic implemented in the `calculateLoad()` method.

3.4.2 Conflict Detection Insights

The error table reliably detected all forms of time conflicts across campuses. For instance, dual assignments of the same instructor in both Abu Dhabi and Al Ain during the same slot triggered the expected conflict messages. Furthermore, missing course offerings on one campus generated accurate warnings. These results demonstrate the robustness of the `detectConflicts()` logic.

3.4.3 Version Control Comparison

The version comparison tool effectively tracked changes across saved schedules. Sessions added, removed, or reassigned were clearly identified and listed, validating the utility of this tool for iterative planning. However, the feature's responsiveness depends heavily on the volume of session data—performance may need optimization for very large datasets.

3.4.4 Export options

Exporting the schedules to Excel and using a calendar API to make the schedules is useful for the user experience. Moreover, it cuts the time necessary to confirm the calendar and copy their data to Excel.

3.4.5 Procedure Quality and Recommendations

Overall, the testing procedure demonstrated strong alignment with expected behavior. Manual testing of each feature confirmed that the system met its design objectives. Still, some improvements are recommended: Implement input validation to reduce user errors during session entry. Sending a message to the authorities for actions concerning the organization. Add AI that can give directions and interact with the users. These enhancements would make the system more resilient and user-friendly for broader deployment.

4 Conclusion

4.1 Summary

This report presented the design and development of a cross-campus course scheduling application tailored for Electrical and Computer Engineering students. In terms of implementation, The system successfully delivered every feature that it intended to: session management, conflict detection, faculty load tracking, and version comparison. Firebase was used for data persistence and collaboration as well as for real-time updates. The results verified that the application delivers on its promise to make it easier to complete tasks related to scheduling, minimizing the number of errors and increasing visibility. The final project can be considered as a scalable solution that could facilitate budgetary planning of academic resources in multi-campus institutions.

4.2 Future Improvements and Takeaways

In this project, we taught ourselves how to build an app in the real world with Angular and Firebase and how to face practical scheduling problems in an academic context. In this process, we have learned a lot about user interface design, real-time data handling, and conflict resolution logic. These principles where applied to a collection of parts to form a functional system that should make a difference. Drag and drop scheduling and enabling real-time collaborative scheduling, where multiple users can edit the schedule simultaneously with conflict resolution and live updates, are our suggestions for improvement. In general, the project helped us to improve our technical

skills and the understanding of collaboration in the software development with regards to an educational context.

4.3 Lessons Learned

In this project, we gained hands-on experience with Angular and Firebase, both of which are completely alien technologies to us. We learned how to create dynamic user interfaces, use real-time datasets, and authenticate. In addition, we have learned many things about preparing for different platforms where imagination is needed, as we cannot see the result on each device when the development stage comes. We also investigated workload balancing, error detection logic, and so on. We have learned the importance of iterative testing and debugging. There are a few challenges, such as resolving scheduling conflicts and ensuring the UI is responsive. These lessons helped us understand full-stack application development in an engineering context.

4.4 Team Dynamics

We are a team of three having our team leader, Zekarias Jemberu. He showed great leadership by co-ordinating tasks, making clear goals and ensuring consistent communication among all members. Through decisions making during key development phases, he kept the project on schedule. To this end, we have created a sort of collaborative and inclusive environment by using a WhatsApp group to communicate on a daily basis and holding weekly meetings in the university lab and library where we work. There was active engagement and participation of all members in the different as- pects of the project like UI design, backend development, testing and report writing as well. The aim of making the programming course scheduling system our main goal is to make it reliable and easy for users to use. The measurable objectives are to use all five core features, do the Firebase integration, and get the testing and documentation done before the deadline. We used a Gantt chart to plan and monitor progress throughout the project. The entire system meets all objectives, and the final system meets the intended performance and functionality.

4.5 Impact Statement

The developed scheduling system is beneficial for society because it helps in academic planning, reduces the work of administrators, and also reduces the chances of making mistakes in making the timetable. Moreover, actual offices are not needed, so paper efficiency is maintained, which also supports environmental sustainability. It is software-based, so there is no electronic waste risk and a small energy requirement. Firebase Authentication is secure, so we need not worry about the privacy of app users. The system allows people to focus on more important work, that is, using their skills and talents. In general, this solution is a more economical means of learning and saves essential resources while cleaning up the environment.

References

[1] R. L. Tokheim, *Digital Electronics: Principles and Applications*, 9th ed. McGraw-Hill Education, 2020.

[2] D. D. Givone, *Digital Principles and Design*, 2nd ed. McGraw-Hill Education, 2019.

[3] B. S. Blanchard and W. J. Fabrycky, *Systems Engineering and Analysis*, 6th ed. Pearson, 2022.

[4] M. Hazzard and M. Hazzard, *Firebase Essentials: Build and Deploy Real-Time Apps*. Leanpub, 2021.

[5] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2020.